

INFORME DE SEGURIDAD Y ESTRUCTURA DEL SISTEMA

Sistema de Gestion y Punto de Venta (POS)

Version del informe: 1.0

Fecha de analisis: 12 de agosto de 2026

Clasificacion: Confidencial - Uso interno

Este documento explica, con palabras sencillas, cuales son los puntos debiles de seguridad y de organizacion interna del programa, y que se puede hacer para mejorar cada uno. Esta pensado para que cualquier persona del negocio pueda entenderlo, sin necesidad de ser experta en computacion.

Como leer este informe:

ROJO: problema grave. Conviene corregirlo lo antes posible.

NARANJA: problema importante. Hay que tenerlo en agenda.

AMARILLO: mejora recomendada para ir mas seguros.

RESUMEN EJECUTIVO

Piense en el sistema como una caja fuerte digital donde el negocio guarda sus datos mas valiosos: ventas, clientes, cuentas corrientes, precios y backups. La buena noticia: la caja fuerte existe y esta bastante bien construida (usa buenas practicas de base de datos y de guardado de contraseñas). La mala noticia: la llave esta pegada con cinta en la puerta, y hay varias puertas traseras abiertas que permiten entrar sin permiso.

Cantidad de hallazgos por nivel de importancia:

ROJO - Critico (corregir ya)	5 hallazgos
NARANJA - Importante (corregir pronto)	6 hallazgos
AMARILLO - Mejora recomendada	5 hallazgos
Estructura - Mejoras de organizacion	9 hallazgos
Fortalezas detectadas (lo que esta bien)	5 puntos

En pocas palabras

- ? Las contraseñas de la base de datos estan escritas en los archivos del programa (visibles en el historial de cambios).
- ? Hay paginas a las que se puede entrar sin iniciar sesion y que pueden modificar la base de datos.
- ? Una pantalla de ventas ejecuta texto sin limpiar (permite ataques desde el navegador).
- ? Los respaldos con TODA la informacion estan guardados en la carpeta publica del sitio, sin proteccion.
- ? No hay cifrado (https) ni protecciones de sesion; las contraseñas viajan como texto plano por la red.
- ? La estructura interna mezcla todo (codigo, disenio y pantalla) en archivos enormes y repetidos, lo que hace caro y riesgoso hacer cambios.

PARTE A - SEGURIDAD DEL SISTEMA

A1. Problemas criticos (ROJO)

CRITICO

1. La contraseña de la base de datos esta escrita en el codigo

Donde:

config/db_config.php, config/db_config copy.php, procesos/cerrar_cajas_historicas.php y hash.php.

Que pasa:

El acceso a la base de datos (usuario "root", el mas poderoso, y su contraseña) esta escrito dentro de los archivos del programa y fue subido al control de versiones (git). Cualquiera que tenga una copia del codigo, o acceda al historial de cambios, ve la contraseña.

Por que es un riesgo:

Si alguien la consigue, puede entrar a la base de datos y ver, cambiar o borrar absolutamente todo: ventas, clientes, cuentas, precios y usuarios. Es la llave maestra de todo el negocio.

Que hacer:

1) Cambiar la contraseña actual. 2) Crear un usuario de base de datos con permisos limitados (no "root"). 3) Mover esos datos a un archivo de configuracion secreto fuera del codigo (ej. un archivo .env que no se sube a git). 4) Quitar del historial de git los archivos que contienen las contraseñas.

CRITICO

2. Paginas "secretas" a las que se entra sin iniciar sesion

Donde:

ejecutar_migraciones.php, diagnostico_cierre_sistema.php, hash.php y procesos/cerrar_cajas_historicas.php.

Que pasa:

Hay archivos del programa que se pueden abrir directamente desde el navegador sin pedir usuario ni contraseña. Algunos de ellos pueden cambiar la estructura de la base de datos (agregar o borrar columnas y tablas), y otros muestran datos internos.

Por que es un riesgo:

Es como dejar la puerta del fondo abierta: cualquiera que encuentre la direccion exacta puede entrar y modificar el sistema desde afuera.

Que hacer:

1) Proteger estos archivos pidiendo sesion iniciada y permiso de administrador. 2) Mejor aun: moverlos fuera de la carpeta del sitio web o eliminarlos. 3) Eliminar hash.php (es una herramienta de desarrollo que ademas muestra una contraseña).

CRITICO

3. La pantalla de ventas ejecuta texto del usuario sin limpiarlo (XSS)

Donde:

pages/ventas.php (linea 804).

Que pasa:

Cuando hay un error de stock, la pagina devuelve al carrito un texto que llega del formulario sin limpiarlo. Ese texto se inserta directamente en el codigo de la pagina.

Por que es un riesgo:

Un texto "trampa" escrito con intencion podria ejecutar codigo en la computadora de cualquier empleado que use el sistema (por ejemplo, robar su sesion, cambiar precios o instalar programas).

Que hacer:

Nunca insertar datos del usuario directamente en la pagina sin "limpiarlos" (escaparlos). En PHP se usan funciones como htmlspecialchars() o json_encode() para que el texto se muestre como texto y no como codigo.

CRITICO**4. La clave secreta de la consulta remota esta a la vista****Donde:**

api/proveedores.php, api/consignaciones.php y en el código visible de pages/consulta_consignaciones_remota.php.

Que pasa:

La clave (token) usada para la consulta remota de proveedores esta escrita dentro de la propia pagina web. Cualquier persona puede verla con solo mirar el código fuente del navegador. Además, la API se puede llamar desde cualquier sitio web (permisos CORS abiertos).

Por que es un riesgo:

Es como tener la contraseña del candado pegada al lado del candado. Un sitio web malicioso podría usar esa clave para leer los datos de proveedores, y la clave no se puede cambiar sin tocar el código.

Que hacer:

1) Usar un acceso con usuario y contraseña de verdad (o un token que el navegador no muestre). 2) Restringir desde que sitios se permite la consulta. 3) Exigir https para esa consulta.

CRITICO**5. Cualquier empleado puede explorar todos los archivos del servidor****Donde:**

ajax/explorador_archivos_backup.php.

Que pasa:

La herramienta que permite elegir la carpeta de backups solo exige "estar logueado". Un vendedor o cajero puede navegar por todo el disco del servidor y crear carpetas en cualquier lugar.

Por que es un riesgo:

El sistema muestra el contenido de todo el servidor (archivos, carpetas, tamaños, fechas) y permite escribir carpetas. Puede exponer información confidencial o dañar la organización de archivos.

Que hacer:

1) Permitir solo a usuarios con rol de administrador o permiso específico de backups. 2) Limitar la navegación a la carpeta de backups. 3) Quitar la opción de crear carpetas.

A2. Problemas importantes (NARANJA)**ALTO****6. Los respaldos (backups) estan en la carpeta publica del sitio****Que pasa:**

Los backups se guardan dentro de la carpeta del sitio web, que cualquier visitante puede intentar abrir desde el navegador. Un backup contiene TODA la información del negocio. Además, la "clave de respaldo" esta escrita en el código (backup_security_key_2024) y el respaldo se imprime entero en el navegador cuando se genera.

Que hacer:

1) Guardar los backups en una carpeta que el navegador no pueda ver (fuera de la carpeta del sitio). 2) Proteger la carpeta con un archivo de reglas (por ejemplo .htaccess) que niegue el acceso. 3) Cambiar la clave de respaldo y guardarla en secreto. 4) No imprimir el contenido del backup en la pantalla.

ALTO**7. Descarga de archivos con ruta controlable por el usuario****Donde:**

pages/backup.php (seccion de descarga).

Que pasa:

La direccion de descarga se arma combinando datos que llegan del navegador. Con un poco de ingenieria se podria intentar leer archivos fuera de la carpeta de backups.

Que hacer:

Validar siempre que el archivo a descargar este dentro de la carpeta de backups y usar nombres generados por el sistema, nunca la ruta que envia el navegador.

ALTO**8. Formularios sin "firma antifalsificacion" (CSRF)****Que pasa:**

Solo dos paginas usan un token de proteccion (abm_empresa y abm_empresas). El resto de los formularios (guardar configuracion, ejecutar backup, cerrar caja, anular ventas, etc.) no verifican que el pedido venga de un empleado real.

Por que es un riesgo:

Un sitio web malicioso abierto en la misma computadora podria "firmar" acciones en nombre del empleado sin que el se entere (por ejemplo, cambiar la configuracion del backup o anular una venta).

Que hacer:

Agregar un token unico por sesion en todos los formularios y validarlo en cada envio, igual que ya se hace en abm_empresa.php.

ALTO**9. Mensajes de error que muestran datos internos del sistema****Que pasa:**

Cuando algo falla, el programa a veces muestra en pantalla detalles tecnicos: rutas de carpetas, mensajes de la base de datos, nombres de archivos. Las paginas de diagnostico tambien exponen este detalle.

Por que es un riesgo:

Esos detalles le "cuentan" a un atacante como esta construido el sistema y donde estan las cosas, facilitando el ataque.

Que hacer:

Mostrar al usuario un mensaje generico ("ocurrio un error, avise al administrador") y guardar los detalles en un registro (log) interno que solo lea el tecnico.

ALTO**10. Sesiones sin protecciones de seguridad****Que pasa:**

Al iniciar sesion no se renueva la identificacion de la sesion (permite "secuestro" de sesion), las cookies no tienen activadas las opciones seguras (HttpOnly, Secure, SameSite), y no hay limite de intentos de contrasena fallidos.

Por que es un riesgo:

Un atacante podria probar contrasenas miles de veces (fuerza bruta) sin ser frenado, o intentar aprovechar una sesion ya iniciada.

Que hacer:

1) Renovar el ID de sesion al entrar (session_regenerate_id). 2) Marcar las cookies como HttpOnly, Secure y SameSite. 3) Agregar limite de intentos fallidos con demora o bloqueo temporal.

ALTO**11. El sistema no usa conexion cifrada (https)****Que pasa:**

El sitio funciona por http, sin el "candado" del navegador. La contraseña y los datos viajan como texto plano por la red.

Por que es un riesgo:

En una red compartida (como la del local o una wifi publica), otra persona podria "escuchar" y leer la contraseña del empleado al entrar.

Que hacer:

Instalar un certificado SSL/HTTPS y configurar el sistema para que solo funcione con https. Tambien se puede forzar la redireccion de http a https.

A3. Mejoras recomendadas (AMARILLO)**MEDIO****12. Subida de imagen del logo con validacion debil****Que pasa:**

La subida del logo solo verifica la extension del archivo (.jpg, .png, etc.) pero no su contenido real ni su tamaño.

Que hacer:

Validar tambien el contenido (tipo MIME real), el tamaño maximo y, si es posible, reprocesar la imagen con la libreria del servidor.

MEDIO**13. Archivos de prueba y copias viejas subidos al repositorio****Que pasa:**

Hay copias de configuracion (db_config copy.php), versiones viejas de paginas (resumen_ventas_viejo.php), multiples versiones del generador de tickets y decenas de archivos de prueba (probar_backup, test_backup, verificar_*).

Que hacer:

Limpiarlos, y agregar reglas al .gitignore para que no se vuelvan a subir. Tambien conviene excluir del repositorio los archivos de configuracion y las carpetas sensibles.

MEDIO**14. El sistema distingue desarrollo y produccion por la direccion web****Que pasa:**

Segun si la URL contiene "_dev", el sistema se conecta a una base de datos o a otra. Si la carpeta cambia de nombre, el sistema puede conectarse a la base equivocada.

Que hacer:

Usar una configuracion explicita por entorno (un archivo .env o un switch de configuracion) en lugar de depender del nombre de la carpeta.

MEDIO**15. El estado de la caja depende de la sesion del navegador****Que pasa:**

La caja abierta queda asociada a la sesion del navegador. Si la computadora se apaga de golpe o se cierra la pestania, el estado puede quedar inconsistente.

Que hacer:

Registrar la caja en la base de datos con hora de apertura/cierre reales y ofrecer un mecanismo de recuperacion.

MEDIO**16. Sin registro de auditoria de acciones importantes****Que pasa:**

No hay un registro central que diga quien hizo que (quien borro un backup, quien cambio un precio, quien anulo una venta).

Que hacer:

Guardar un registro (log) de las acciones criticas con fecha, usuario y detalle, para poder revisar ante cualquier duda.

PARTE B - ESTRUCTURA INTERNA DEL SISTEMA

B1. Lo que esta bien (fortalezas)

- ? Las consultas a la base de datos usan "sentencias preparadas" (PDO). Esto es la forma correcta de evitar la inyeccion SQL, el ataque mas comun contra sistemas web. Es una base solida.
- ? Las contraseñas se guardan cifradas con el metodo recomendado (bcrypt / password_hash).
- ? Existe un sistema de permisos por empleado y por rol (vendedor, cajero, supervisor, administrador).
- ? El sistema soporta varias empresas y sucursales, y en general filtra los datos por empresa. Es una buena arquitectura para crecer.
- ? Hay una carpeta de migraciones que documenta los cambios de estructura de la base de datos, lo que ayuda a mantener orden historico.

B2. Debilidades de estructura (como esta armado)

MEDIO

1. Archivos gigantes que mezclan todo

Donde:

Por ejemplo pages/ventas.php tiene alrededor de 965 lineas con HTML, CSS, JavaScript y logica de negocio juntos.

Que pasa:

Cada pantalla es un archivo enorme donde se mezcla la "cara" del sistema (diseno), la "cabeza" (reglas de negocio) y el "corazon" (acceso a datos).

Por que importa:

Cualquier cambio pequeno puede romper otra cosa sin darse cuenta. Es dificil de mantener, de revisar y de probar.

Que hacer:

Separar por capas: una capa de presentacion (la pantalla), una capa de logica de negocio (las reglas) y una capa de acceso a datos.

MEDIO

2.Codigo duplicado

Donde:

Hay dos archivos de configuracion de base de datos, varias versiones del generador de tickets (ticket_generator, temp_ticket_generator, ult_ticket_generator, funciona_ticket_generator), y paginas viejas que ya no se usan (resumen_ventas_viejo).

Que pasa:

El mismo codigo esta copiado en varios lugares con pequenas diferencias.

Por que importa:

Si se corrige un error en una copia y no en la otra, el error sigue vivo. Ademas genera confusión sobre cual es la version "buena".

Que hacer:

Dejar una sola version de cada cosa, mover el codigo comun a funciones compartidas y eliminar lo que no se usa.

MEDIO**3. Diseño (CSS) repetido en cada página****Que pasa:**

Los estilos visuales están copiados dentro de cada página en lugar de estar en archivos comunes.

Por que importa:

Para cambiar un color o un botón hay que tocar decenas de archivos, con riesgo de olvidar alguno y que el sistema se vea desparejo.

Que hacer:

Centralizar los estilos en archivos .css comunes (o un tema) y quitarlos de las páginas.

MEDIO**4. No hay pruebas automáticas****Que pasa:**

El sistema no tiene pruebas que se ejecuten solas para verificar que todo sigue funcionando.

Por que importa:

Al hacer un cambio no hay red de seguridad: puede romperse algo importante (por ejemplo el cálculo de una venta) y nadie se entera hasta que falla en el mostrador.

Que hacer:

Empezar con pruebas para las partes críticas (cálculo de ventas, cuentas corrientes, cierres de caja) y correrlas antes de cada cambio.

MEDIO**5. Las migraciones de la base se aplican a mano****Que pasa:**

Los cambios de estructura se hacen con scripts sueltos que hay que ejecutar a mano, en el orden correcto.

Por que importa:

Si se saltea una o se ejecuta dos veces, la base puede quedar en un estado raro.

Que hacer:

Usar un sistema de migraciones que sepa cuáles ya se aplicaron y las aplique en orden automáticamente.

MEDIO**6. No se usa un gestor de dependencias completo****Que pasa:**

La única librería externa administrada es la de AFIP. El resto del código, las herramientas y los estilos están "a mano".

Que hacer:

Usar Composer para las librerías de PHP y npm para las de JavaScript, así las actualizaciones y copias en otra máquina son sencillas.

MEDIO**7. Sin registro central de errores y acciones****Que pasa:**

Los errores se vuelcan en la pantalla o se pierden; no hay un lugar único donde ver que falló y cuando.

Que hacer:

Centralizar el registro de errores en archivos de log con fecha, página y detalle, y revisarlos periódicamente.

MEDIO**8. Archivos y carpetas huérfanos de desarrollo****Que pasa:**

Hay páginas de diagnóstico, scripts de prueba y archivos temporales en la carpeta del sitio que no forman parte del sistema productivo.

Que hacer:

Mover todo el material de desarrollo fuera de la carpeta pública o a un repositorio separado.

MEDIO**9. No hay una separación clara entre "página visual" y "acción interna"****Que pasa:**

Los procesos (guardar, anular, cerrar caja) están mezclados con las pantallas, y los accesos por AJAX son muy variados en cuanto a cómo validan permisos: unos piden rol, otros permiso, otros solo "estar logueado".

Que hacer:

Crear un punto de entrada único y una función estándar de control de acceso que todos los procesos usen (ya existe el helper `require_permiso`, conviene aplicarlo en todos lados).

PARTE C - PLAN DE ACCION SUGERIDO

Fase 1 - Urgente (hacer esta semana)

- ? Sacar las contraseñas del código y cambiar la contraseña actual de la base de datos. Crear un usuario de base de datos con permisos limitados.
- ? Proteger o eliminar las páginas a las que se entra sin sesión (migraciones, diagnostico, hash.php, cierre masivo).
- ? Mover los backups fuera de la carpeta pública y bloquear el acceso web a la carpeta de backups.
- ? Corregir el problema del carrito de ventas (XSS) y restringir el explorador de archivos a administradores.
- ? Revisar el historial de git para quitar los archivos con contraseñas (y evitar que vuelvan).

Fase 2 - Corto plazo (1 a 3 semanas)

- ? Activar https (certificado SSL) y forzar su uso.
- ? Agregar tokens antifalsificación (CSRF) a todos los formularios.
- ? Ocultar los mensajes de error detallados y volcarlos a un registro interno.
- ? Sesiones: renovar el ID al entrar, cookies con opciones seguras y límite de intentos de contraseña.
- ? Cambiar el acceso a la consulta remota: clave que no esté en el navegador y restringir desde donde se puede llamar.

Fase 3 - Mediano plazo (1 a 3 meses)

- ? Separar el código en capas (pantalla / lógica / datos) y crear funciones compartidas.
- ? Eliminar archivos duplicados y páginas viejas; centralizar los estilos en archivos comunes.
- ? Crear pruebas automáticas para las partes críticas.
- ? Implementar un sistema central de migraciones y un registro de auditoría.
- ? Configuración por entorno (.env) en lugar de depender del nombre de la carpeta.

CIERRE

El sistema tiene buenas bases: usa sentencias preparadas, contraseñas cifradas y un esquema de permisos. Los problemas más urgentes no son de "rehacer todo", sino de tapar las puertas abiertas: sacar las contraseñas del código, proteger las páginas de acceso libre, guardar bien los backups y activar https. Con la Fase 1 se elimina la mayor parte del riesgo real.

Nota: este informe fue generado a partir de la revisión del código del proyecto. Los nombres de archivos